Numerical Project in Python n.2: Semidefinite embedding

4OPT2 - Continuous Optimisation

Nathanaël SEROPIAN & Ramzi JEBALI *February-March 2026*

1 Introduction

The core idea of this project is to reconstruct the spatial geometry of a set of locations using only their pairwise distances. To achieve this, we do not directly search for the coordinates, but rather for an intermediate Gram matrix X^* such that $X_{ij} = \langle x_i, x_j \rangle$.

We solve the following Semidefinite Programming (SDP) problem, minimizing $-\text{trace}(X)$ subject to physical and geometric constraints:

- $1_n^\top X 1_n = 0$, which ensures the point cloud is perfectly centered at the origin, otherwise the solution would be unbounded and the solver would diverge;
- $X_{ii} + X_{jj} - 2X_{ij} = D_{ij}, \quad \forall i, j$, which mathematically translates the physical distances into our Gram matrix elements, enforcing the preservation of distances;
- $X \succeq 0$, which is the fundamental condition ensuring X is a valid Gram matrix.

The objective function, minimizing $-\text{trace}(X)$, is equivalent to maximizing the trace. Physically, the trace represents the sum of the squared norms ($\sum \|x_i\|^2$), meaning we are maximizing the variance of the point cloud. This acts as a "Maximum Variance Unfolding" process: the solver stretches and unfolds the structure as much as possible without violating the fixed pairwise distances, naturally revealing the minimal required dimension to embed the data.

To extract the coordinates, we perform a spectral decomposition of the optimal Gram matrix X^*. The eigenvalues λ_i of X^* indicate the embedding dimension d.

2 Semidefinite Embedding

Question 1: Solving the SDP and dimensionality

Numerically, we observe a clear structural gap in the eigenvalue spectrum: exactly 3 eigenvalues are strictly and significantly positive, while the remaining $n - 3$ eigenvalues are exceptionally small (restricted only by the solver's numerical precision limit, e.g., $\sim 10^{-7}$). Therefore, the minimal embedding dimension is exactly $d = 3$.

More precisely, the matrix X obtained from the SDP is a Gram matrix, hence it can be factorized as $X = K^\top K$. Writing the spectral decomposition $X = V \Lambda V^\top$ with $\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_n)$, we obtain $K = \Lambda^{1/2} V^\top$. The rows of K therefore represent the coordinates of the cities in Euclidean space. Since there are three eigenvalues which are not close to zero, the points lie in a 3-dimensional subspace.

This result provides a definitive mathematical proof regarding the shape of the data: if the Earth were flat, or if it were a cylinder (which can be unrolled without metric distortion), the distances would perfectly embed in a 2-dimensional space, yielding exactly 2 non-zero eigenvalues ($d = 2$). Because we strictly require $d = 3$ to unfold the data while respecting the geographic distances, we can confidently conclude that the Earth is not flat.

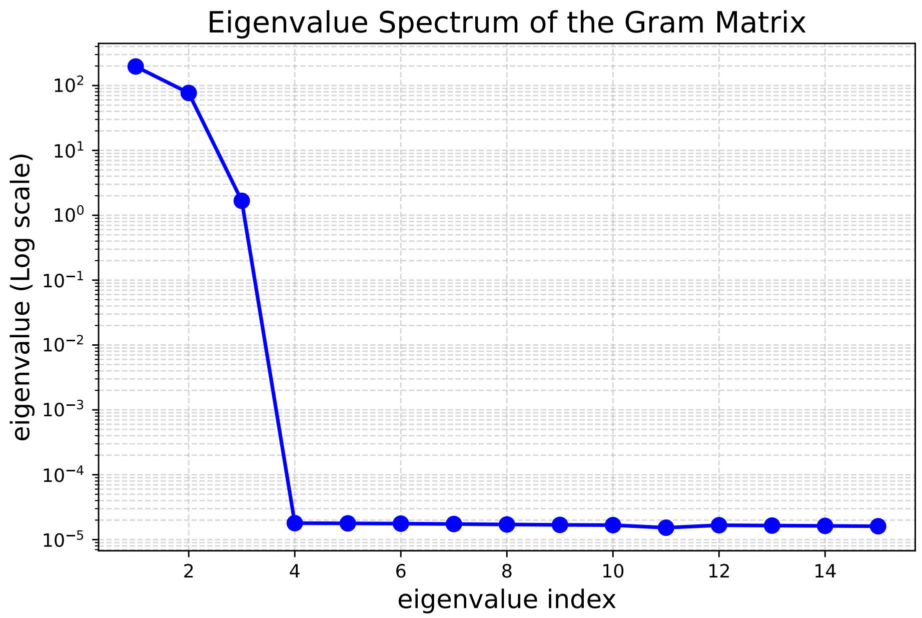

Figure 1: Eigenvalue spectrum of the optimal Gram matrix X^* (logarithmic scale). The sharp drop after the 3rd eigenvalue visually confirms that the minimal embedding dimension is strictly $d = 3$.

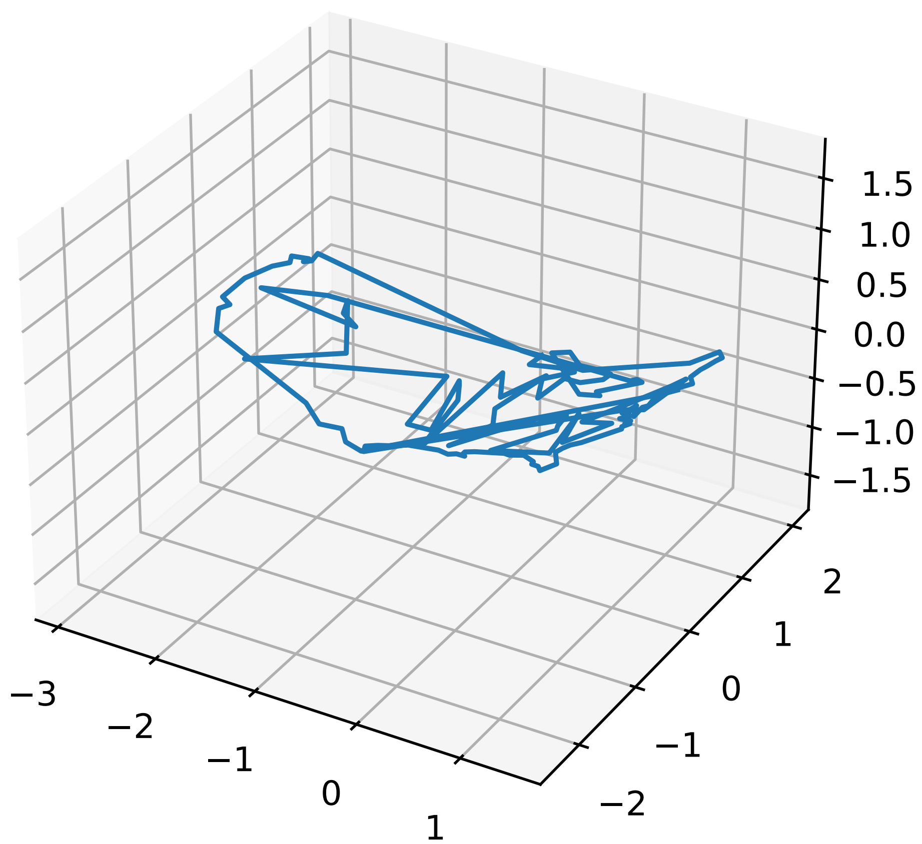

Figure 2: Three-dimensional embedding of the US cities dataset. The optimal Gram matrix naturally unfolds the pairwise distances into a 3D curved surface, visually confirming the non-flat geometry.

Question 2: Sparsification and computational complexity

When moving to larger datasets, such as `D60` or the full matrix `D` ($n = 7800$), the default Interior-Point Methods (IPM) used by solvers like `cvxpy` become too slow.

Let n be the number of locations. The number of strict distance constraints is $m = \frac{n(n-1)}{2} = \mathcal{O}(n^2)$. In a standard IPM for SDP, constructing and solving the KKT linear system at each iteration requires a computational complexity ranging from $\mathcal{O}(m^2 n^2)$ to $\mathcal{O}(m^3)$. For the full dataset ($n = 7800$), the number of constraints m reaches approximately 30×10^6. The sheer size of the resulting KKT matrix vastly exceeds the RAM limits of standard computers.

To mitigate this bottleneck, we implemented a sparsification strategy, retaining each distance constraint with a Bernoulli probability $p = 0.2$. This effectively reduces the number of constraints to $\tilde{m} \approx p \cdot m \approx p \cdot \mathcal{O}(n^2)$. This approach is highly effective for medium-sized problems:

- By discarding 80% of the constraints, the memory footprint and the cost of inverting the linear KKT system are drastically reduced, allowing the solver to process intermediate matrices like `D60`.
- Dropping edges might seemingly distort the map, but pairwise distances in a dense graph are highly redundant. Keeping only 20% of the edges creates a sparse but sufficient "rigid skeleton" that maintains the global geometry of the 3D structure, yielding a valid embedding.

However, despite this sparsification, the time complexity remains polynomial with a very large exponent. For the full $n = 7800$ dataset, even a sparse subset of $\approx 6 \times 10^6$ constraints remains too large. This highlights a fundamental limitation of second-order optimization methods and justifies the need for first-order splitting algorithms —such as the Douglas-Rachford splitting— which operate with significantly lower memory requirements per iteration.

Question 3: Scaling up with Douglas-Rachford Splitting

To bypass the memory bottleneck of second-order methods, we implemented the Douglas-Rachford Splitting (DRS) algorithm. We reformulate the SDP as minimizing the sum of two convex functions:

$$\min_{X \in \mathbb{S}^n} f(X) + g(X), \tag{1}$$

where $g(X) = \iota_{\mathcal{S}_+^n}(X)$ is the indicator function of the Positive Semi-Definite (PSD) cone, and $f(X) = -\mathrm{trace}(X) + \iota_{\mathcal{C}}(X)$, with $\mathcal{C}$ representing the affine constraints (centering and distances).

The proximal operator of g, denoted prox_g, trivially consists in diagonalizing the matrix and thresholding any negative eigenvalues to zero. The true mathematical challenge lies in prox_f.

By definition, we have (using a canonical step size $\gamma = 1$):

$$\mathrm{prox}_f(Z) = \arg\min_{X \in \mathcal{C}} \left(-\mathrm{trace}(X) + \frac{1}{2}\|X - Z\|_F^2 \right). \tag{2}$$

As we know that $\mathrm{trace}(X) = \langle I, X \rangle_F$, let us expand the quantity $\frac{1}{2}\|X - (Z + I)\|_F^2$:

$$\begin{aligned} \frac{1}{2}\|X - (Z + I)\|_F^2 &= \frac{1}{2}\|X - Z\|_F^2 - \langle X - Z, I \rangle_F + \frac{1}{2}\|I\|_F^2 \\ &= \frac{1}{2}\|X - Z\|_F^2 - \langle X, I \rangle_F + \langle Z, I \rangle_F + \frac{1}{2}\|I\|_F^2 \\ &= \left(\frac{1}{2}\|X - Z\|_F^2 - \mathrm{trace}(X) \right) + C, \end{aligned} \tag{3}$$

where $C = \mathrm{trace}(Z) + \frac{1}{2}n$ is a constant independent of X. Since adding a constant does not alter the location of the minimum, the proximal operator simplifies to:

$$\mathrm{prox}_f(Z) = \arg\min_{X \in \mathcal{C}} \frac{1}{2}\|X - (Z + I)\|_F^2. \tag{4}$$

This result demonstrates that maximizing the trace under geometric constraints is strictly equivalent to shifting the input matrix Z by the identity matrix I, and then performing an orthogonal projection of $M = Z + I$ onto the affine subspace $\mathcal{C}$.

To compute this projection without inverting a massive KKT system, we applied the **Dual Ascent method** on the dual Lagrangian formulation:

$$\mathcal{L}(X, \lambda) = \frac{1}{2}\|X - M\|_F^2 + \langle \lambda, \mathcal{A}(X) - D \rangle_F, \tag{5}$$

where $\mathcal{A}(X)$ is the linear operator mapping a Gram matrix to pairwise distances, and λ is the matrix of dual multipliers.

Minimizing the primal yields $X^*(\lambda) = M - \mathcal{A}^*(\lambda)$. The adjoint operator $\mathcal{A}^*(\lambda)$ is the **Graph Laplacian** of λ, defined algebraically as $\mathrm{Diag}(\lambda \mathbf{1}) - \lambda$.

The dual variables are iteratively updated using gradient ascent: $\lambda_{k+1} = \lambda_k + \alpha(\mathcal{A}(X_k) - D)$. Based on convex optimization theory, to guarantee unconditional stability, the step size α must satisfy $\alpha < 2/L'$, where L' is the Lipschitz constant of the dual gradient. Spectral analysis of the operator $\mathcal{A}\mathcal{A}^*$ bounds this constant at $L' \leq 4n$. So we set $\alpha = \frac{1}{4n}$.

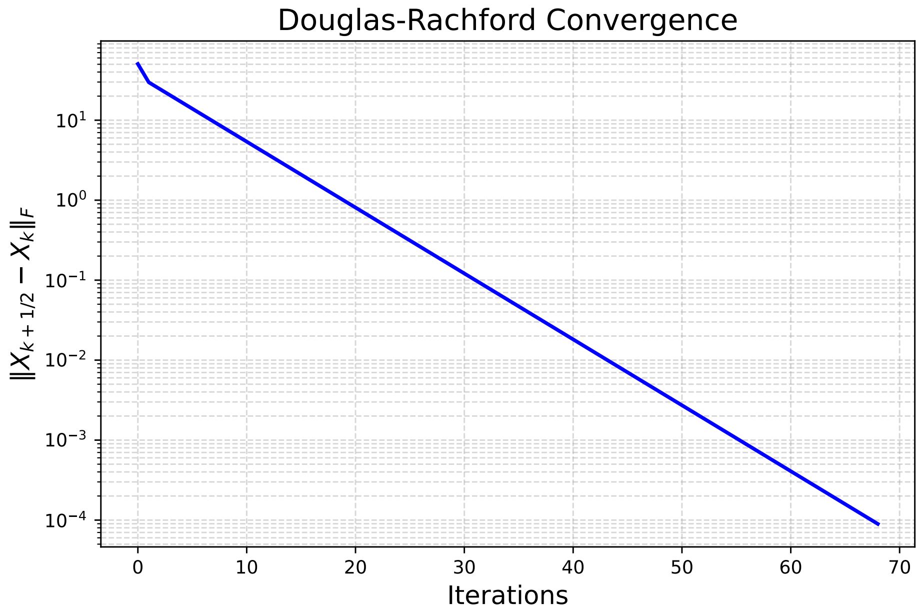

Figure 3: Convergence of the Douglas-Rachford splitting algorithm. The strictly monotonic decay of the fixed-point residual confirms the absolute stability provided by our theoretical step size $\alpha = 1/(4n)$.

Remark on the Proximal Step Size in Financial Applications: we implemented the Douglas-Rachford algorithm without a step size (effectively $\gamma = 1$), such as in the first project. However, in industrial applications—such as quantitative finance— introducing a tunable proximal step size $\gamma > 0$ is standard practice. For instance, when applying SDP to *Covariance Matrix Calibration* or *Robust Portfolio Optimization*, the scale of the objective function and the scale of the constraints often differ by several orders of magnitude. In such ill-conditioned problems, γ acts as a crucial **preconditioning parameter**. Tuning γ balances the primal and dual residuals, vastly accelerating the convergence rate from hundreds of iterations to merely a few dozen, which is vital for low-latency algorithmic trading systems.

Question 4: Estimating the Earth's Radius

Having successfully embedded the locations in a 3-dimensional space ($d = 3$), we obtain the geometric coordinates $x_i \in \mathbb{R}^3$ for each city. Our next goal is to leverage these physical coordinates to estimate the Earth's radius, denoted by R.

At this stage, a critical geometric observation must be made to avoid a common conceptual trap: due to the centering constraint $\sum_{i=1}^{n} x_i = 0$ enforced during the SDP optimization, the origin $(0, 0, 0)$ of our coordinate system is strictly the barycenter of the continental USA points. It is *not* the center of the Earth. Therefore, calculating the norm $\|x_i\|$ will not yield the radius. Instead, we must fit a sphere of unknown center $C \in \mathbb{R}^3$ and unknown radius R to our resulting 3D point cloud.

The mathematical equation of the sphere for each point x_i is given by: $\|x_i - C\|_2^2 = R^2$.

Expanding the squared Euclidean norm, we obtain a non-linear relation in C and R:

$$\|x_i\|_2^2 - 2x_i^\top C + \|C\|_2^2 = R^2. \tag{6}$$

To linearize this problem, we introduce a scalar variable $y = R^2 - \|C\|_2^2$. The equation can then be rewritten as a linear system:

$$2x_i^\top C + y = \|x_i\|_2^2, \quad \forall i \in \{1, \dots, n\}. \tag{7}$$

We formulate this as a linear least-squares minimization problem. Let $A \in \mathbb{R}^{n\times 4}$ be the matrix whose i-th row is $[2x_i^\top, 1]$, and let $b \in \mathbb{R}^n$ be the vector with entries $b_i = \|x_i\|_2^2$. We solve for the optimal parameter vector $v = [C^\top, y]^\top \in \mathbb{R}^4$:

$$v^* = \arg\min_{v\in\mathbb{R}^4} \|Av - b\|_2^2. \tag{8}$$

Once the optimal center C^* and the scalar parameter y^* are extracted from v^*, we seamlessly deduce the Earth's radius:

$$R = \sqrt{\|C^*\|_2^2 + y^*}. \tag{9}$$

Numerically, using our embedded coordinates, the least-squares solver computes an estimated radius of $R \approx 6.371$. As specified in the dataset documentation, the unit of measurement is 1000 km. Therefore, our model yields a final radius of **6371** km.

This result matches the widely accepted volumetric mean radius of the Earth with astounding precision. Such an outcome brilliantly validates the geometric fidelity of our Semidefinite Embedding approach: by strictly preserving local pairwise distances, the optimization problem inherently reconstructs the exact global curvature of the underlying manifold (the sphere) without any prior geographic knowledge.

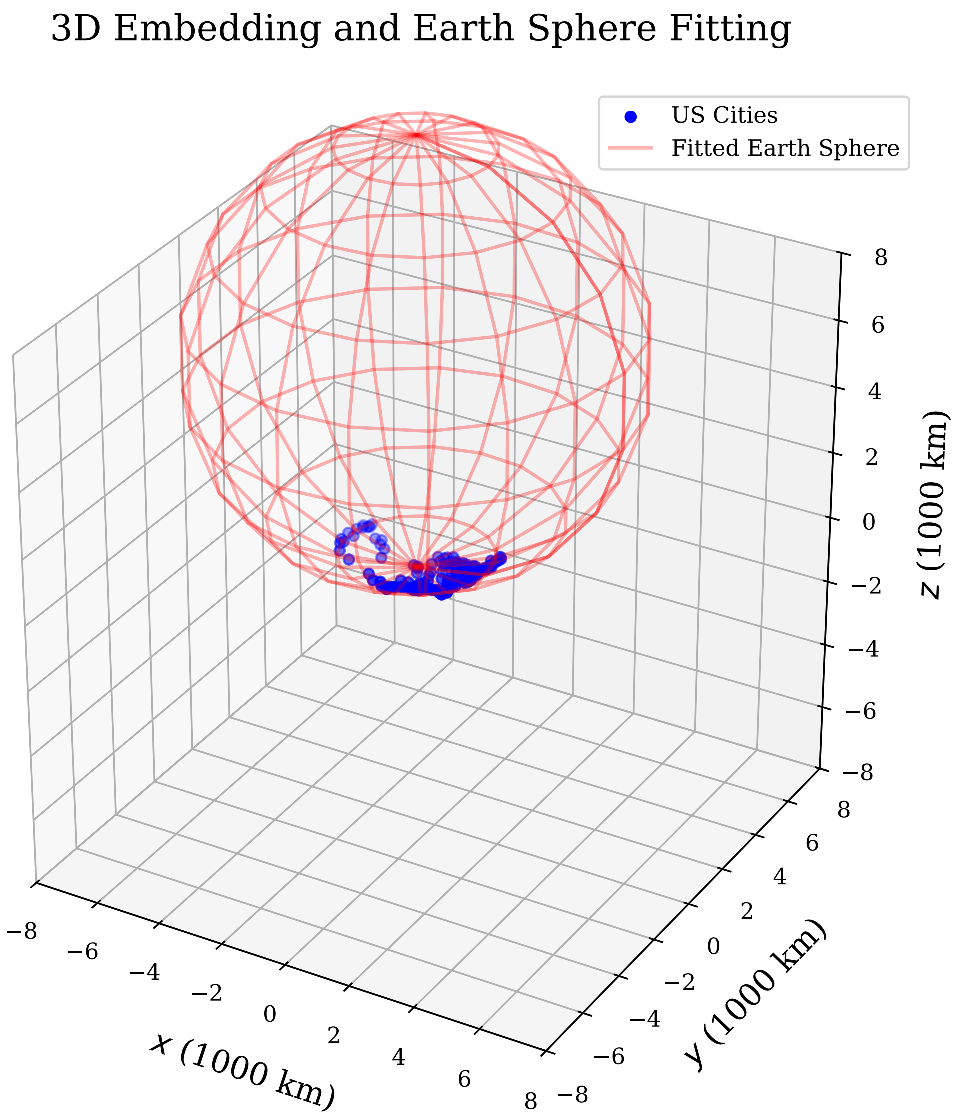

Figure 4: The embedded 3D locations fitted to a theoretical sphere using the linearized least-squares method. The derived center and radius reconstruct the full Earth's globe from purely local distance data.